MEAD: Mamba-based Environment for Automated Linux Command Deception Framework Using an Adversarial Shell Command Dataset

Anonymous Author(s)

Abstract

As attackers become more proficient, traditional honeypots face challenges in sustaining realistic interactions and incur high environment-specific configuration costs. While recent Large Language Model (LLM)- and Reinforcement Learning (RL)-based deception frameworks have attempted to address this gap, they can suffer from state inconsistency and noticeable response latency, thereby deviating from real-world interactive shell behaviors. In this paper, we present an Mamba-based Environment for an automated Linux command deception framework (MEAD), which enables lightweight cyber deception by generating Linux command-line outputs without deploying a fully emulated deception environment. MEAD represents each step of an attacker's session as a structured context consisting of the current command, recent history, and a compact environment state; then it utilizes a Mamba State-Space based Model (SSM) to generate the corresponding shell output. To support training and evaluation, we constructed a Linux command dataset that maps commands and environment states to output across multi-step attack sequences. We evaluate MEAD based on output fidelity and runtime behavior across diverse command sequences, demonstrating that it can generate plausible, state-consistent responses while improving responsiveness compared to heavyweight generative deception baselines. Compared with open-weight LLM baselines (Qwen, GPT-OSS, and Gemma), MEAD achieves the best output fidelity as measured by Exact Match, Character F1, and ROUGE. Beyond output fidelity, MEAD provides resource efficiency advantages and broader interaction coverage under long command histories.

1 Introduction

Improved network accessibility and computer service diversity offer numerous advantages to users, but paradoxically, the advancement of computer systems increases the attack surface within the operating environment [38]. The expansion of the attack surface has led to increasingly complex and dynamic threat landscapes, making it challenging for security operators to predict attacker behavior and identify potential attack targets. As adversarial behavior becomes increasingly unpredictable, security systems require proactive and adaptive defense mechanisms beyond traditional rule-based approaches. Security operators have continuously devised various defensive techniques under constrained resources, including cyber deception mechanisms such as honeypots.

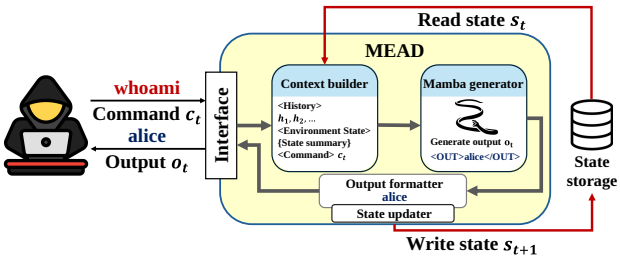


Figure 1: High-level overview of MEAD framework.

Honeypot, among cyber deception techniques, has been proposed for various domains to protect their assets by using a fake environment comparable to their operating environment [33]. Honeypots deceive attackers and collect malicious behavior data, enabling security operators to design effective countermeasures for their environments. However, a honeypot requires manual configuration for each deception framework to obtain suitable malicious information that utilizes the actual operating environment. As honeypot technologies become widespread, attackers attempt to identify and evade deployed honeypots; security designers have to anticipate potential attacker behaviors and design highly realistic deception environments to deceive adversaries effectively. The limited ability of traditional honeypots to deceive attackers highlights the need to predict malicious behavior and design more effective deception strategies.

Given these constraints, cyber deception has evolved beyond traditional honeypots into an environment-adaptive defense framework that integrates deceptive environments with

System	Shell Emulation	Stateful Tracking	Hallucination Mitigation	Inference Efficiency	Real-time Response	Context Scalability
Adaptive LLM Context Honeypot [29]	GPT-3.5-turbo	✓	✓	✘ ($O(L^2)$)	✘	△
HoneyGPT [45]	GPT-3.5-turbo, GPT-4	✓	✓	✘ ($O(L^2)$)	✘	△
shelLM [31]	GPT-3.5-turbo	✓	✓	✘ ($O(L^2)$)	✘	✘
LLM Honeypot [26]	Llama3 8B	△	△	✘ ($O(L^2)$)	✘	△
HoneyLLM [11]	GPT-3.5-turbo, GPT-4, Claude, Llama2	✓	✓	✘ ($O(L^2)$)	✘	△
SPADE [2]	ChatGPT-4o, ChatGPT-4o Mini, Gemini, Llama3.2	✘	✓	✘ ($O(L^2)$)	✓	△
VelLMes [32]	GPT-3.5-16k, GPT-4	✓	✓	✘ ($O(L^2)$)	✘	△
MEAD	Mamba	✓	✓	✓ ($O(L)$)	✓	✓

Table 1: Comparison of existing cyber deception frameworks.

behavioral information of attackers. The capabilities of cyber deception enhance the prediction of adversarial behavior, reducing attack effectiveness by compelling adversaries to spend additional time and resources within the deception environment. However, the diversity of attack techniques, the attacker's awareness of the deception environment, and evasion greatly affect the attacker's behavioral patterns, making it challenging to construct a cyber deception framework that reacts to complex patterns using only simple rule-based analysis. The attacker's erratic behavior has facilitated a paradigm shift from static honeypots to intelligent deception frameworks capable of adaptive interaction using artificial intelligence. Notably, recent cyber deception frameworks have proposed the use of Transformer-based Large Language Models (LLMs) that can interact with attackers without having to implement a real systematic environment [2, 29]. LLM-based cyber deception frameworks emulate the environment, reproducing interactions in a terminal environment and generating outputs corresponding to various attack techniques used by attackers [11, 26, 31, 32, 45]. In parallel, Reinforcement Learning (RL) and Game Theory optimize resource orchestration and adapt to the intelligent attacker's behavior [7, 9, 20, 46]. Existing studies have formalized and understood the strategic interactions of attackers and developed optimal approaches to deceive and disrupt [23, 27, 42, 47].

Despite the progress in intelligent deception, existing Transformer-based and RL-based frameworks face critical structural limitations. **First**, the quadratic computational complexity ($O(L^2)$) of the attention mechanism in Transformers leads to significant memory bottlenecks as command sequences lengthen. As an attacker's session progresses, the increasing context size causes memory exhaustion and performance degradation, making it difficult to sustain long-term engagement without compromising state consistency. **Second**, LLM-based systems suffer from inherent inference latency that hampers real-time deception. The time-to-first-token delay in large models often exceeds the immediate response expectations of a standard Linux shell. **Third**, RL-based models often struggle with stability and limited interaction coverage. As noted by Bridges et al. [3], these systems can produce unrealistic responses during early-stage training or fail to adapt to unpredictable attacker behaviors outside their training distribution. Furthermore, the issue of 'hallucination' remains prevalent in LLMs, where the model fails to maintain a logically coherent file system state, a vulnerability that identification tools like TRAP can exploit [12].

These challenges necessitate a shift toward more efficient sequence modeling. The selective State-Space models (SSMs) introduced in Mamba [10] offer linear-time complexity ($O(L)$) and superior memory recall. In this paper, we propose a Mamba-based Environment for the Automated Linux Command Deception (MEAD) framework that generates suitable outputs in response to input commands entered by an attacker in a Linux environment. Following the table 1, the MEAD framework provides a lightweight cyber deception environment by synthesizing command outputs via a generative model, eliminating the need for security operators to construct and manage a full-scale systematic environment compared to other existing models. Figure 1 demonstrates the basic functions of the MEAD framework. The MEAD framework generates realistic Linux shell outputs for an attacker's commands while maintaining consistency with the stashed command history and the environment state. Notably, we specifically focused on the Linux environment, widely used as a server platform, on which both administrators and attackers rely on the same shell interface to compromise systems [13]. To verify the aforementioned methodologies, the MEAD framework required the dataset that corresponds to the attack command and output results produced by the attacker over time, as well as the environmental state at the time the attack command was input. However, no dataset is presented that maps the output values corresponding to the attack commands performed by the attacker. Thus, we created a Linux shell command dataset that maps outputs corresponding to the attacker's input commands. The constructed dataset contains a series of attack command sequences, along with the corresponding environment state, including the timestamp, username, description, and working directory. The Mamba model uses each environmental statement to understand the flow of the attack sequence and derive relevant results.

The main contributions of this paper are as follows:

- We propose MEAD, an interactive and lightweight cy-

ber deception framework that generates state-consistent Linux shell outputs for an attacker's command inputs without deploying and maintaining a complete physical decoy system.

- We introduce an efficient training and inference methodology for long and variable command outputs that addresses output fidelity and ground-truth leakage.
- We create a state-annotated Linux command–output dataset and a practical model adaptation strategy to enable robust domain specialization under limited data and constrained training budgets.

2 Background

2.1 Honeypot and Cyber Deception

A honeypot is a decoy system designed to simulate a vulnerable environment, allowing defenders to lure, monitor, and analyze attacker behavior without exposing real assets. By isolating malicious activity in a controlled environment, honeypots provide valuable insights into adversarial tools, tactics, and attack patterns with minimal risk to operational systems [28]. Honeypots can be deployed across various services, ranging from Secure Shell (SSH) to full-scale web servers, and the collected attack data can be leveraged to enhance the security of production systems [15]. Honeypots are commonly classified into low-, medium-, and high-interaction types based on the level of system exposure they provide [40]. Low-interaction honeypots offer limited functionality and do not grant access to a real operating system (OS), exposing only predefined attack surfaces. Medium-interaction honeypots emulate partial system behavior and can execute certain commands, enabling observation of more complex attack techniques without deploying a full OS. In contrast, high-interaction honeypots employ real OS or applications to capture detailed attacker behavior, though they require substantial maintenance and pose higher operational risks

Cyber deception extends the concept of honeypots into a broader defensive framework that employs deceptive environments and false information to proactively detect, mislead, and analyze adversarial actions [16]. Unlike traditional static or rule-based honeypot systems, cyber deception aims to adopt adaptive and automated mechanisms that respond dynamically to attacker behavior. However, due to the non-deterministic and evolving nature of cyber threats, static deception approaches still face fundamental limitations, motivating the integration of AI-driven techniques to enhance adaptability and realism [21, 43]. To address these challenges, recent AI-based research explores simulation and camouflage strategies, such as mimicry and disguise, to create uncertainty and divert attackers toward non-critical or decoy assets [22].

Despite these advances, many existing honeypots and early cyber deception frameworks continue to inherit static and rule-based design principles. Following the long-term surveys about classification of honeypots indicate that even as these frameworks have been extended to diverse environments such as IoT and cyber-physical systems, their core design principles remain largely unchanged [8, 33, 48].

Such static characteristics introduce deterministic behavior patterns that can be exploited through fingerprinting techniques. Early studies demonstrated that low- and medium-interaction honeypots are susceptible to detection due to predictable responses [24]. This limitation has been empirically validated at scale, where consistent protocol handshakes and infrequent updates enable attackers to reliably distinguish honeypots from real hosts [34, 39]. Similar identification techniques have also been observed in specialized domains such as industrial control systems, where discrepancies between emulated environments and physical hardware expose deception systems [49].

These findings highlight that static and rule-based deception mechanisms struggle to sustain high-fidelity interaction against automated and intelligent reconnaissance, motivating the development of more adaptive and dynamic deception frameworks.

2.2 State-space Model

SSM is a class of algorithms that mathematically expresses the relationship between the three elements of a dynamical system: input, state, and output. SSM is a general framework that can represent nonlinear and time-varying systems, including Linear Time-Invariant systems (LTI). Also, SSM has the superposition feature where both state changes and outputs can be expressed as linear combinations of the current state x and input u. In cases where the system coefficients do not depend on time, the SSM exhibits time-invariance, meaning that its governing equations retain the same structure across all time steps. SSM can determine a system's dynamic properties using superposition and time-invariance characteristics, generally represented by a state equation and an output equation [41].

Equation 1 represents the state equation. The state equation expresses the state vector $x(t)$ at time t and the rate of change of the state vector $\dot{x}(t)$ through the external signal $u(t)$ given to the system at time t [44]. A is the state transition matrix, which determines how the current state changes over time. B is the input matrix, which represents the effect of external input on state changes. Equation 2 represents the output equation. The output equation determines the output vector $y(t)$ through the state vector $x(t)$ at time t and the external signal $u(t)$ given to the system. The output equation describes how the internal state of the system is observed externally, and expressed as a linear combination of the given input $u(t)$ and the current state $x(t)$. The C represents the output matrix that illustrates the projection of the internal state onto external observations. D is a feedthrough matrix, which represents the path through

which the input immediately affects the output.

$$\dot{x}(t) = Ax(t) + Bu(t) \tag{1}$$

$$y(t) = Cx(t) + Du(t) \tag{2}$$

In summary, SSM provides a general mathematical framework for structurally analyzing and controlling complex dynamic systems by expressing the system's operating process as changes in internal states and the resulting outputs. However, it struggles to capture nonlinear sequence patterns or complex contextual interactions because SSM used in deep learning commonly relies on linear state transitions. Moreover, state transitions over time are defined as linear accumulation; as the input length increases, past information becomes diluted, making it challenging to maintain contextual information. The state vector $x(t)$ of SSM mainly depends on local information around the current point in time, and as a result, it has the limitation of being challenging to maintain the entire input history in a consistent structure in complex time series or memory-based tasks.

To address these limitations, Mamba, a selective state update structure that extends conventional linear SSMs to be input-conditional, was presented. Mamba maintains the linear dynamics of the state-space layer, but uses a selective scanning method that dynamically adjusts the state transition matrix and input coupling matrix through a gating function based on the input u_k. Gating is an input-conditioned modulation mechanism that adjusts the information flow based on the significance of the input; it determines the state update rate at time point k through the gate value of $g_k \in [0,1]$. Mamba generates gate values from input u_k, which are defined by equation 3. The generated gates are combined with the basic SSM parameters to form a dynamic filter as in equation 4. Mamba performs input-dependent state updates as in equation 5.

$$g_k = \sigma(W_g u_k) \tag{3}$$

$$A_k = g_k \odot A, \qquad B_k = g_k \odot B \tag{4}$$

$$x_k = A_k x_{k-1} + B_k u_k \tag{5}$$

$A_k(u_k)$ and $B_k(u_k)$ are dynamic parameters that depend on the input, and $A_k(u_k)$ is a state transition matrix dynamically determined by the input u_k at time k. $B_k(u_k)$ means the input combination matrix dynamically generated based on the input u_k at time k. $A_k(u_k)$ and $B_k(u_k)$ adjust the strength and direction of state update according to the contextual information of the input, unlike the existing fixed $\bar{A}$ and $\bar{B}$. Mamba uses a selective filtering architecture to achieve performance comparable to or better than Transformers on language modeling and time series data for long sequences, without relying on self-attention. Moreover, since Mamba can significantly reduce memory usage and inference latency, this paper employs Mamba to implement a cyber deception framework requiring low memory usage and real-time inference.

In this paper, we present MEAD, a lightweight cyber deception framework built on Mamba that provides realistic Linux shell outputs conditioned on command history and environment state. Notably, MEAD reduces overhead by leveraging selective state updates and enabling low-latency, state-consistent responses suitable for interactive deception.

3 Methodology of MEAD

We analyzed the command input values and attack behavior patterns entered by the attacker using natural language processing. Attackers required a series of behaviors to achieve the attacker's goals, such as asset theft or system deterioration, including accessing, hijacking, and analyzing the victim's computer [17]. In other words, cyber attacks do not occur through a single isolated action but instead require a structured sequence of attack behaviors. Detecting and preventing only a single attack event within a honeypot is insufficient; it does not entirely control the attack and provides no insight into the attacker's behavior, making an effective response impossible. Notably, attackers continuously interact with the victim environment, adjusting their actions based on the system state and the commands they ordered. The attacker's step-by-step interaction has the characteristic that the results of each step are directly accumulated and reflected in the decisions and actions of the next step. Therefore, we tried to understand the contextual data of commands between the attacker and victim environments and to understand the attacker's procedural behavior.

Figure 2 shows an overview of the MEAD framework. MEAD consists of a training process and an inference and interaction process. The training process consists of a data preparation process that processes the command-output dataset as the Mamba model's input, and a hybrid training process trains the Mamba model based on the converted sequence. In the data preparation process, each command, output, and state at the time the attacker executed the command are tagged in the command-output pair dataset. We tokenize the tagged data so that each token receives a numeric ID for use by the Mamba model. Then perform the sanity check to ensure that each command–output pair is correctly tagged and contains no structural errors. In the hybrid training process, train the model using the processed command–output pairs as input. We train a Mamba-based model on a domain-specific dataset composed of structured command–output sequences, enabling the model to infer system outputs in real time based on an attacker's input commands and the corresponding environment state. In our framework, the Mamba model captures the relationships between diverse environmental states and security command contexts, providing powerful generalization across varied attack scenarios, which improves effectiveness and stability in cyber deception environments. Our approach addresses the context-maintenance limitations and response delays common in conventional LLM-based methods; it fur-

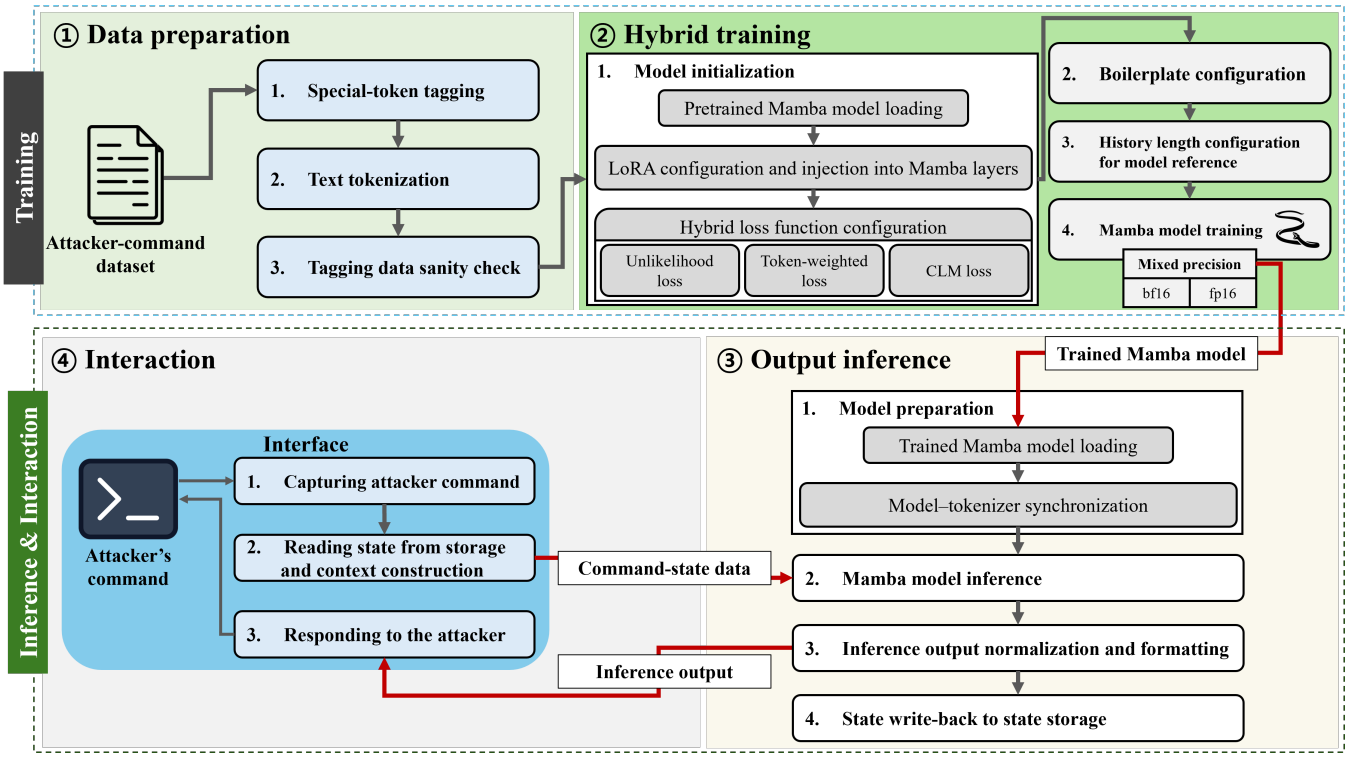


Figure 2: The overall structure of MEAD framework. MEAD consists of two phases: a training process and an inference-interaction process. In the training phase, MEAD converts an attacker-command dataset into a machine-readable sequence that jointly represents the environmental state and the attacker's command history; trains a Mamba-based generative model to map command context to shell output. In the inference-interaction process, MEAD captures each incoming attacker command in real time, updates the internal state, and constructs a context sequence including the configured history length to condition the trained Mamba model. The model then generates the output for the given command, and MEAD returns the generated output to the attacker, enabling interactive, shell-consistent deception without deploying a physical Linux environment.

ther avoids the initial training inefficiency and environment-adaptation constraints seen in RL-based approaches, enabling the real-time predictive performance and robustness required in cyber deception environments.

To utilize the properties of the Mamba model in cyber deception, we construct a structured command–output representation that enables the model to learn how attack behaviors unfold within a given environment. First, we converted the attacker's command history into a single structured text sequence consisting of a series of command–output pairs. Then we tokenized the converted text sequence and vectorized sequence to train the Mamba model so that it can learn the mapping between commands and their corresponding outputs. The trained model receives a specific Linux command under a referenced environmental state and generates the potential system output automatically. The trained Mamba model processes the attacker's command sequence and the fake environment state as a unified context and produces the output that interacts with the attacker in a real cyber deception setting. During training, we define frequently appearing expressions as boilerplate and apply unlikelihood-loss-based suppression so that the model does not overuse common phrases or repetitive patterns and produce incorrect outputs. We also created a hybrid loss structure that includes Causal Language Modeling (CLM) loss, token-weighted loss, and unlikelihood loss, allowing the model to learn accurate outputs for each command. Then, the trained model is deployed in the inference and interaction process to generate state-conditioned outputs in real time.

The inference and interaction process includes the interaction process that engages with the attacker and the output inference process that generates the appropriate output for the attacker's input during interaction. In the interaction process, return the output generated by the trained model immediately whenever the attacker accesses the fake cyber deception environment and executes a command, enabling continuous interaction. When the attacker sends a command to the fake cyber deception environment, the interaction process builds the current state and the input command as context, then relays them to the output inference process. In the output inference process, the trained Mamba model references the current state, generates the output for the input command, and returns the

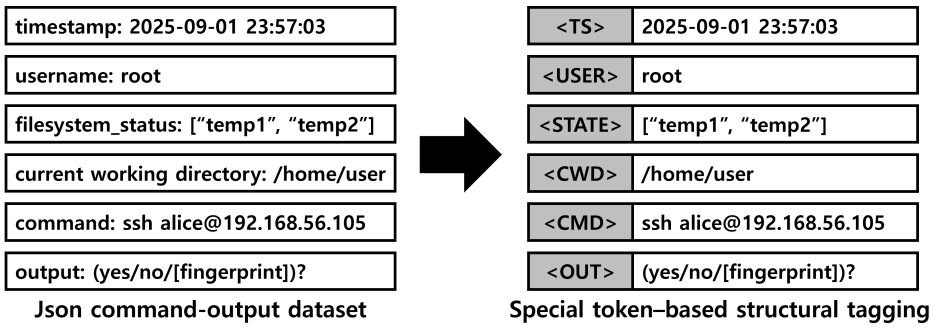


Figure 3: Example of tagging process that performs mapping each command–output pair using a set of special tokens.

result to the interaction process. The trained Mamba model, fine-tuned through Low-Rank Adaptation (LoRA), predicts the output that may occur at each step based on the tagged state information and the input command. In this paper, we load the model parameters in 4-bit quantized form to reduce latency after receiving the attacker's input. The interaction process updates the modified state with the result and sends the generated output to the attacker. This paper utilized mixed precision based on bfloat16 (bf16) and float16 (fp16) to improve model learning and inference efficiency. fp16 and bf16 are numeric formats, which are the way floating-point numbers are represented on a computer. fp16 has the advantage of improving learning speed by reducing memory usage and computational load, but has the problem of a relatively narrow value range. The bf16 refers to a 16-bit floating point format, which has the advantage of having a wide range of expressible numbers and being resistant to gradient underflow, where the operation result converges to 0. We utilized mixed precision to accelerate learning while ensuring numerical stability and preventing model performance degradation.

3.1 Data Preparation and Input Standardization

We conduct data preparation and input standardization to use command-output pairs as training data for the Mamba model.

3.1.1 Tagging Process

First, we tag each command–output pair using a set of special tokens so that the model can understand the input as a structured sequence instead of free-form text.

Figure 3 illustrates an example of the tagging process. We transformed JSON format command-output data into an understandable format for the model using the predefined tags. We assign auxiliary tags to each special token so that each command–output pair takes contextual information at execution time. Based on the auxiliary tags, we implement the model to treat the task as conditional generation, where equivalent commands can generate different outputs under different execution contexts, rather than to memorize an input–target mapping simply. Notably, we explicitly separate the input section and the target section using dedicated tags to support stable training. Table 2 summarizes the special tokens used for tagging and describes the meaning of each token.

We named the smallest unit instance for training and evaluation in the tagging process as a sample. Each sample consists of five blocks: header, context, reference, prompt, and output. The header block contains information demonstrating the execution time and the executed command, including the delimiter `<SEP>` that separates each sample, and a `<SEP_SAMPLE>` tag that separates the execution context within the sample. If we merely tag the attacker's command-output pair, there is a limitation that the Mamba model cannot produce appropriate output even if it reflects the changed state. For example, when an attacker creates a file in a specific directory and then attempts to execute it, Mamba model still outputs that the file does not exist if it relies only on fragmented context, even when it receives the updated state information. In this paper, we set the history length L of each command so that the model could sufficiently refer to previous commands to output suitable output. History length means the nearest description of interactions that occurred before the current prediction target command-output pair, which is included in the model input. A sample includes not only the current command–output pair but also a history that accumulates previously executed commands and their outcomes; the model trains that the same command can generate different outputs depending on prior state changes. The sample represents history through reference tags that provide prior interactions in a structured form, and is composed of reference blocks. Through experiments, we set the history length to 4 to balance the performance and cost of command-output generation. Reference blocks list the most recent L interactions according to history length L, allowing the model to reference previous contexts reliably. By isolating the reference information into a separate block, the model is less likely to misperceive prior records for the current correct answer range, or the boundary between input and correct answer is strengthened. The history uses `<REF_CMD>` to denote a previous command, `<REF_SIG>` to summarize salient

Token	Category	Description
<TS>	Structural	Indicates the command execution timestamp.
<CMD>	Structural	Marks the command line.
<SEP>	Structural	Delimits sections within a sample.
<SEP_SAMPLE>	Structural	Delimits samples.
<PROMPT>	I/O Boundary	Marks the model input interval and defines the input range during training.
<TARGET>	I/O Boundary	Marks the ground-truth target segment.
<USER>	Metadata	Indicates user and account identifiers.
<CWD>	Metadata	Identifies the current working directory.
<STATE>	Metadata	Indicates system state.
<META>	Metadata	Indicates other metadata besides the current tag.
<HOST>	Metadata	Indicates the host machine identifier.
<ENV>	Metadata	Indicates Environment variables and environment information.
<REF_CMD>	Reference	Provides prior commands in history to condition the current prediction.
<REF_SIG>	Reference	Provides compact reference signals and signatures useful for conditioning.
<REF_STAT>	Reference	Provides reference status and state summaries to support state-aware generation.
<OUT_BEGIN>	Output Delimiter	Specifies the start of the output text.
<OUT_END>	Output Delimiter	Specifies the end of the output text.
<OS>	System	Indicates the OS identification information section.
<UID>	System	Indicates the UID.
<GID>	System	Indicates the GID.
<SID>	System	Indicates the SID.
<EOT>	EOS	A single-token EOS marker that is used consistently for both sequence termination and padding.

Table 2: Special tokens used for tagging command-output pairs and associated metadata. The tokens designate a structured input format, explicitly separating prompt and target segments and delimiting the output range for reliable generation.

characteristics of that command's output, and <REF_STAT> to represent state changes or key state information, enabling the model to reference earlier context reliably. Within a sample, the <SEP_SAMPLE> token separates individual history entries.

Also, the sample contains a context block that specifies the conditions at the time of command execution, such as the user account, current working directory, and system status. The context block guides the model to consider not just the command, but also execution time conditions when generating the output. The context block records <USER> to represent the user account identifier, <CWD> to denote the current working directory, <STATE> to summarize the system state, <HOST> to specify the host identifier, <ENV> to capture environment variables and execution-environment information, and <META> to include additional metadata. The context block also records <OS> to describe OS identification information, <UID> to represent the user ID, <GID> to represent the group ID, and <SID> to represent the session ID.

The prompt block is the area that specifies the current command input that the model should predict. The prompt block displays the model's input as <PROMPT> so that the model can clearly distinguish the command entered by the attacker; we indicate the command line within the prompt block as <CMD>. Additionally, we specified the scope of the generated text of the command execution result corresponding to the prompt through the output block. To effectively process variable-length outputs generated according to input commands, we distinguished the start and end of the output based on the output delimiter. Additionally, by utilizing the <TARGET> tag, the boundary between input and correct answer was set by classifying the output according to the command. We normalized all sequences to end with a single token <EOT>.

3.1.2 Tokenization Process

We performed subword tokenization under a fixed tokenizer configuration, ensuring a consistent token-to-ID mapping for reliable training and inference of tagged command–output sequences. First, we adopted subword tokenization to represent command–output text as a sequence of subword units. A sub-

word tokenizer is a method of breaking down text into meaningful subword units and converting them into tokens [19]. The word-based tokenizer tokenizes all words in the training data to create a vocabulary and conducts integer encoding based on the vocabulary. However, when the model uses a word-based tokenizer, it faces a situation where new word tokens that are not in the training data, Out Of Vocabulary (OOV), can arise. When OOV occurs, the language model stumbles to understand the meaning and context of the sequence accurately [50]. Subword-based tokenization methods have the advantage of alleviating the OOV problem and representing strings with periodic mutations more reliably. Notably, commands contain diverse and high-entropy string patterns such as paths, identifiers, hashes, and option flags; subword tokenization provides stable segmentation for such patterns and allows preserving their structure during modeling. We applied a subword tokenizer based on the Byte Pair Encoding (BPE) algorithm, which finds the pairs of letters that appear most frequently in succession and iteratively merges the most frequent adjacent symbol pairs to form larger subword units [30].

Additionally, we used a fixed tokenizer to preserve the same vocabulary during training and inference. Fixed tokenizer refers to using the same token set, special token definitions, and normalization rules established during the training phase throughout the inference process. If the tokenizer changes during the inference process, the tokenization result and ID may change even for the exact string, which can lead to loss of special token meaning or performance degradation. To address this problem, we configured to store the tokenizer used during the training phase and load the same tokenizer during inference. Furthermore, we fixed the registration order for special tokens so that the tokens used for tagging consistently map to the same ID.

3.2 Hybrid Training

MEAD is not only to generate output according to commands, but also to enable the model to reliably train conditional generation problems where output can vary even for the same command depending on history and context. We applied CLM-based implementation and LoRA-based finetuning technique to effectively apply tagged command-output sequences to Mamba-based models.

3.2.1 CLM with Target-only Label Masking

To generate output corresponding to the input command, we implemented a Mamba-based generative model based on the CLM methodology. CLM means a method of predicting the next token x_t given all tokens $x_{1:t-1} = (x_1, \ldots, x_{t-1})$ that appeared before point t in the sequence. Since the output depends on the input command, we need to score each candidate token in the vocabulary and determine which token is the correct next output token at each time step. We used logits z_t, which are values representing the score for each token, to calculate the score for selecting each candidate token v included in the vocabulary V. logits z_t can be expressed as $z_t \in \mathbb{R}^{|V|}$, and represents the scores for candidate tokens in vocabulary V expressed as the set of real numbers $\mathbb{R}$. Then, we converted the scores for each token into a probability distribution based on the softmax function to represent the logits scores as a probability distribution. Equation 6 represents the softmax function.

$$p_\theta(x_t = v \mid x_1, \ldots, x_{t-1}) = \frac{\exp(z_{t,v})}{\sum_{u \in V} \exp(z_{t,u})} \tag{6}$$

The probability of outputting the current token x_t, given the model parameters θ and the previous tokens $x_1, \ldots, x_{t-1}$, is computed by exponentiating the score $z_{t,v}$ of each candidate token $v \in V$ and normalizing it by the sum $\sum_{u \in V} \exp(z_{t,u})$ over all candidate tokens. The Mamba model trains to assign high probabilities to correct tokens, and outputs suitable tokens for the input command through a prompt block consisting of history, context, and command, and softmax. Significantly, we optimized a hybrid loss that combines standard CLM with token-weighted loss and unlikelihood loss. The hybrid loss strengthens training on output-critical sections and discourages undesirable patterns such as boilerplate repetition, which are common in long and variable command outputs.

If the model calculates the loss value by including a prompt block, the model can be optimized to reproduce prompt tokens rather than learning the command-output generation, which can degrade output fidelity. Thus, we configured the training to compute loss only for the correct output block by applying target-only label masking. Target-only label masking sets the labels of all non-target tokens to a designated ignore value and keeps valid numeric IDs only for tokens within the `<TARGET>` tag. Target-only label masking prevents the model from training on masked tokens, so the training process focuses on precise conditional generation without reproducing the input tokens as output. We applied it to effectively process variable output for the input value, command, defined as equation 7.

$$\begin{aligned}
\mathcal{L}_{\text{hybrid}} &= \mathcal{L}_{\text{maskedCLM}} + \lambda_{\text{eot}}\, \mathcal{L}_{\text{UL-eot}} + \lambda_{\text{bp}}\, \mathcal{L}_{\text{UL-bp}} \\
\mathcal{L}_{\text{maskedCLM}} &= -\sum_{t=2}^{T} m_t\, w_t \log p_\theta(x_t \mid x_1, \ldots, x_{t-1}) \\
m_t &= \begin{cases} 1, & t \in \mathcal{T}, \\ 0, & t \notin \mathcal{T}. \end{cases}
\end{aligned} \tag{7}$$

w_t is a token-level weight that enhances the contribution of critical output tokens, including the termination token, during optimization. $\mathcal{L}_{\text{UL-eot}}$ penalizes assigning high probability to the termination token at non-termination target positions, suppressing premature end predictions. $\mathcal{L}_{\text{UL-bp}}$ penalizes tokens that would trigger repetitive line-level boilerplate patterns under the current context. In the process of calculating the

hybrid loss, we determine whether to exclude or not based on the mask $\mathbf{m}_t \in \{0,1\}$ at time t; $\mathcal{T}$ represents the set of token indices corresponding to the target output block within the entire sequence. If the $\mathbf{m}_t$ value is 1, the token x_t at time t corresponds to $\mathcal{T}$, so the loss value is calculated. If the $\mathbf{m}_t$ value is 0, it corresponds to a prompt block composed of other tokens and excluded from the loss value calculation. λ_{eot} and λ_{bp} are non-negative weighting factors that control how strongly the EOT-related and boilerplate-related unlikelihood penalties influence the overall optimization relative to the masked CLM loss. A masked CLM loss alone increases the probability of ground-truth tokens; however, it does not explicitly decrease the probability of undesirable behaviors such as premature end predictions and repetitive boilerplate patterns. By tuning these λ values, we balance faithful token-level learning with behavior-level constraints so that the model remains accurate while avoiding systematic failure modes in long and variable command outputs. When calculating the loss value, we start calculating from point $t = 2$, which refers to the previous token. Based on the target-only label masking method, the model was induced to generate correct output for the given information and input command.

3.2.2 Selective LoRA Targeting for Mamba

In summary, the proposed model trains conditional output generation by providing a prompt block consisting of history, context, and command as a condition and calculating the loss only for output block tokens. However, directly adapting the overall model parameters against variable and long output sequences significantly increases the training cost and increases the possibility of overfitting on limited data. To address this issue, we use a selective fine-tuning strategy that applies LoRA to a subset of weight matrices in a Mamba-based generative model, enabling domain adaptation without updating the full model parameters. Linear transformation weights W map the input representation dimension d_{in} to the output representation dimension d_{out}; the weight matrix is denoted as $W \in \mathbb{R}^{d_{out} \times d_{in}}$ LoRA is a Parameter-Efficient Fine-Tuning (PEFT) technique that trains a low-rank approximation of a pretrained model's weight update instead of updating the full weight matrix W [14]. LoRA learns two low-rank matrices $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$ to construct the weight update ΔW rather than modifying W directly. LoRA defines the effective weight during training as W', as shown in Equation 8.

$$W' = W + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA \tag{8}$$

r denotes the rank and is generally set to satisfy $r \ll min(d_{in}, d_{out})$. α is a scaling factor that controls the magnitude of the LoRA update, and $\frac{\alpha}{r}$ normalizes the update scale across different ranks. Following Equation 8, LoRA reduces memory usage and training cost by optimizing only A and B without updating the complete set of model parameters. We selectively apply LoRA to projection modules involved in gating and state updates within the Mamba model, rather than adapting the entire parameter set, enabling efficient adaptation to command–output patterns. Notably, the proposed command–output generation considers not only surface-level textual similarity but also the impact of accumulated environment state changes induced by command execution. Accordingly, we focus LoRA updates on computations that govern state representation updates and the formation of the output token distribution.

3.3 Inference Strategy

Our inference strategy aims to reliably generate outputs that can deviate depending on command history and environment state, while preventing ground-truth leakage in which ground-truth tokens are inadvertently included in the inference input and the model copies the answer. Since the training data are constructed by concatenating a prompt block and an output block, we use only the prompt block as the Mamba model's input in the inference process and strictly ensure that tokens belonging to the output block are not included in the inference input. Our design prevents the Mamba-based model from observing the ground-truth output and enables conditional generation under realistic interactive settings. In addition, we use the same fixed tokenizer as in training to prevent token-ID drift. During the inference process, the model sequentially generates output tokens based on the token probability distribution when given an input prompt. Output generation terminates when the model produces the `<EOT>` token, and only the range defined as the output block is returned as the final output. Specifically, we extract the output range using `<OUT_BEGIN>` and `<OUT_END>`; if `<OUT_END>` is missing, we return the tokens before `<EOT>`, and apply post-processing to ensure that prompt tags do not pollute the final output. Additionally, we consider that even the same command can produce different outputs depending on changes in history and environmental states. Therefore, inference is not a one-shot generation process; instead, it is executed as an interaction loop in which we construct the prompt from the current environment state and history, generate and return an output, and then update the environment state based on the generated result. By repeating the interaction process, the attacker can continuously receive state-consistent outputs for consecutive commands during step-by-step attack interactions.

4 Experiments

We tested that MEAD generates proper output for the input command based on our self-constructed Linux command-output dataset. Additionally, we compared the metrics of the results of the MEAD with those of open-weight LLM models.

4.1 Experimental Setup

Our experiments were conducted on a desktop PC with one NVIDIA 2080 Super GPU with 8GB of memory. The desktop PC's CPU is an Intel® Core™ i7-9700k. In terms of software, the PC runs on the Ubuntu 24.04.2 LTS OS; the experiments apply Python version 3.12.3, CUDA version 12.0, PyTorch version 2.8.0, and the Mamba model version mamba-2.8b-hf.

Target LLMs We compared the open-weight LLM provided by Ollama to apply the Linux command-output dataset and verified that it produces proper output. We used Retrieval-Augmented Generation (RAG) to generate output related to input commands based on open-weight models such as Gemma3-4b, GPT-OSS-20b, and Qwen2-7b [1, 36, 37]. RAG means a strategy that enables a model to perform proper inference without model tuning by inserting external information into the prompt. We utilize RAG to set up frameworks and compare accuracy without arbitrary manipulation of the usage model. RAG assists inference by inserting retrieved examples as additional context into the prompt, without fine-tuning the model. We apply the same retrieval rules and an identical prompt template across all models to ensure a fair comparison between the proposed framework and baseline accuracy.

Dataset. Our study requires a dataset that supports credible reproduction of terminal outputs for attacker commands under evolving execution contexts. Concretely, the dataset must provide multi-step attacker sessions, where each step contains the command, and an explicit snapshot of the environment state that can affect subsequent outputs, such as the working directory and file system changes. Several existing datasets are closely related to our approaches, but do not satisfy the command output and environment state [4, 18]. Therefore, we constructed our own dataset by executing attacker commands against multiple independently configured victim environments and recording multi-step sessions as command output pairs with explicit environment state transitions. Our dataset enables direct sustenance for output generation and supports controlled evaluation across history lengths by grounding each output in the precise state accumulated from prior commands. By focusing the dataset on outputs and environment state, we ensure that the evaluation reflects the characteristics of state-consistent terminal emulation. The full attacker command dataset is available to researchers upon request under controlled access, with usage restrictions in place to reduce the risk of misuse.

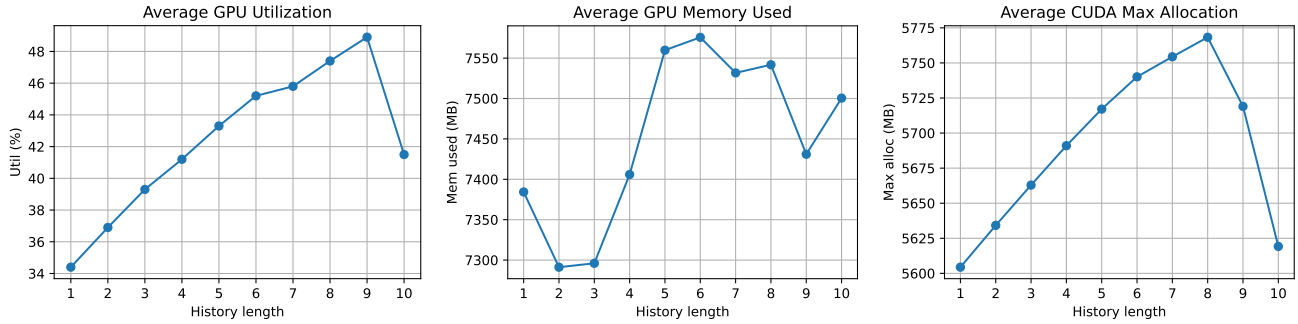
Experimental metrics. The goal of our paper is not simply to output suitable sequences, but to reproduce interactions in real environments by generating output corresponding to the input command. In our experiment, interactions in a specific operating environment can produce various outputs depending on the state of the environment, even with the same input values, so the output does not result in a single correct answer. In the field of artificial intelligence, the confusion matrix is generally utilized to evaluate the model's performance, a classification indicator that summarizes prediction performance. However, the confusion matrix considers a single class prediction; it is difficult to determine whether the generated sequence can be used as an output corresponding to the input command. Therefore, we used Exact Match (EM), Character-level F1 (Char-F1), and Recall-Oriented Understudy for Gisting Evaluation (ROUGE) to evaluate how sufficiently the model-generated output matches the correct answer. We designed the system to measure not only complete matches but also partial matches and phrase-level similarity based on three indicators, enabling a multi-faceted comparison of the suitability of the generated results.

More details of the target LLMs, dataset, and experimental metrics are demonstrated in the appendix.

4.2 Inference Performance

We first tested whether MEAD framework and three open-weight LLM models appropriately generate output corresponding to the attacker's command input. We calculated the average accuracy over 200 epochs for each history length. In appendix, table 6 shows the results of verifying the MEAD and three open-weight LLM models using metrics. The proposed MEAD surpasses open-weight models in conditional generation problems that produce outputs corresponding to commands. Additionally, MEAD achieves higher Char-F1 and ROUGE scores than EM, indicating that even when it fails to reproduce the output exactly, it consistently generates outputs that are highly similar to the reference. For the open-weight LLM baselines, we observed that they can produce outputs that are substantially similar to the reference outputs, even when accounting for the fact that their responses are derived using RAG. However, these open-weight LLMs frequently exhibited hallucinations by generating content that was not present in the retrieved evidence, which degraded the overall quality of the outputs. Notably, all models, including MEAD, showed significant metric changes as the history length changed, and in the case of the MEAD framework, it showed the best performance when history length was 4, in Table 3. As the history length increases, the metric score of the MEAD framework decreases, but the open-weight LLM model maintains a comparable level. The proposed MEAD framework outperforms other models in the EM metric, which calculates a score when all characters are correct, demonstrating that the MEAD framework accurately constructs output.

Also, we calculated the average time to perform the command-output cycle according to the history length to test the real-time response. The time measured in this paper represents the average time from when the command c_t is input by the attacker to when the output o_t for that command is returned to the attacker. MEAD is designed to generate output as quickly as possible, with an average response time of



(a) Average GPU utilization. (b) Average GPU memory usage. (c) Average CUDA max allocation.

Figure 4: GPU statistics by history length. The overall history length ranges from 1 to 10, and we tested GPU utilization, average GPU memory usage, and average CUDA max allocation. The experimental results showed that GPU utilization and CUDA usage increased in proportion to the history length, and unusually, it decreased when the history length was 10. In terms of GPU memory usage, an average of 7,400 MB was utilized, showing that the GPU memory was utilized to the maximum extent.

Model	EM	Char-F1	ROUGE-1	ROUGE-2	ROUGE-L
Gemma3-4b	0.503	0.737	0.656	0.628	0.654
GPT-OSS-20b	0.456	0.537	0.517	0.551	0.517
Qwen2-7b	0.319	0.686	0.584	0.459	0.582
MEAD	0.939	0.954	0.941	0.939	0.940

Table 3: Inference results on each model in history length 4.

1.3372 seconds across the entire history length. Gemma3-4b and Qwen2-7b models' generation time was around 3 seconds, but the GPT-OSS-20b model took an average of 19.952 seconds to generate output. MEAD framework demonstrated an immediate response to shorter inputs, but the time taken to generate outputs increased with the length of the response.

4.3 Resource Efficiency Evaluation

We checked the GPU utilization, GPU memory usage, and CUDA allocation according to the history length to verify whether the proposed framework can generate output within limited resources.

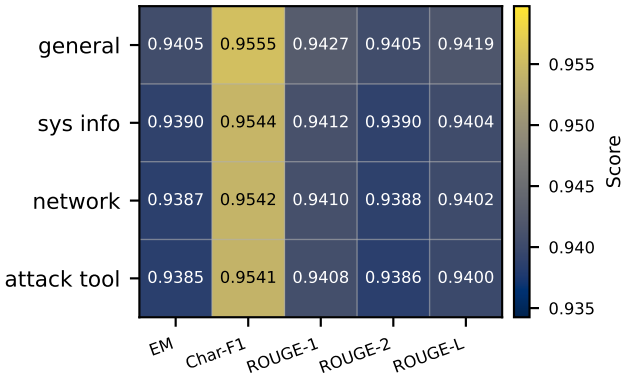
Figure 4 shows GPU statistics over history length. As shown in Figure 4a, the MEAD framework's GPU usage increased as the history length increased, but when the history length exceeded 10, the GPU utilization decreased. As can be seen in Figure 4b, for GPU memory, the highest was observed when the history length was 6, and 7400 MB was utilized when the most optimal history length was 4. Also, as shown in figure 4c, for CUDA allocation, it was highest when the history length was 8, but it decreased sharply when the history length became longer. In conclusion, we have confirmed that our model can effectively generate output even within limited GPU performance.

4.4 Interaction Coverage Evaluation

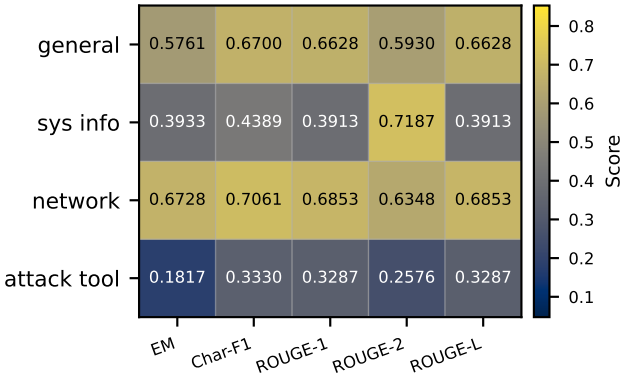
We tested whether MEAD and open-weight LLM interact with various input commands. First, we classified the attacker commands included in the dataset into four types based on existing command taxonomy studies [5, 25, 35]. General commands include common shell usage such as file and directory manipulation, text processing, and system information includes commands to check system status, such as OS, permissions, processes, and hardware. The network consists of communication-related commands, such as network connection verification and scanning, and the attack tool is defined to include commands used in the execution and penetration stages of attack tools, such as Metasploit. Figure 5, the MEAD produces robust output across various categories. Notably, we confirmed that not only is it strong in generating general commands, but it also suitably outputs commands generated by attack tools. The open-weight LLM models are suitable for generating output related to general commands, networks, and system information, but have difficulty generating output corresponding to attack tools. In particular, in the case of Qwen2-7b, it was difficult to generate output related to system information and output related to attack tools. In summary, our proposed MEAD framework can generate outputs corresponding to a wider variety of commands compared to existing open-weight LLM models.

4.5 Comparison with Open-Weight LLMs

A deception framework should adapt to diverse and should adapt to each environment to detect attacks in each cloud environment. Additionally, we mentioned that the cyber deception environment should prevent attackers even without deploying a fully emulated deception environment. Through experiments, we verified that we not only output results corresponding to the attacker's input even without implementing

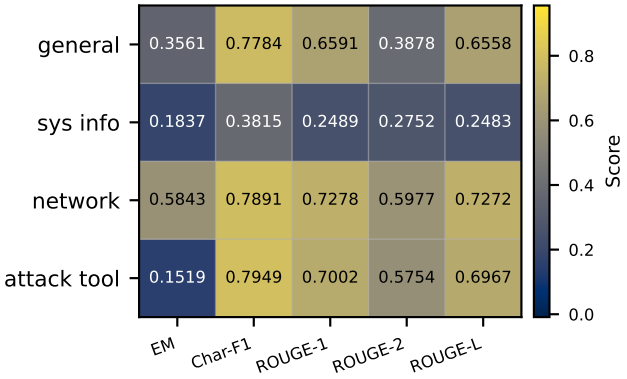


(a) Interaction coverage in the MEAD framework.



(b) Interaction coverage in the GPT-OSS-20b model.

(c) Interaction coverage in the Gemma3-4b model.



(d) Interaction coverage in the Qwen2-7b model..

Figure 5: Overview of interaction coverage in each model. The Mead framework produced all four categories we specified appropriately. The open-weight LLM model also produced appropriate output, but did not produce output corresponding to the attack tool-related commands.

a real environment, but also constructed a lightweight cyber deception environment. Notably, there were differences between the proposed model and the open-weight LLM in the process of generating output according to the attacker's input.

In appendix, table 7 shows examples of output values for each model based on each attacker's input. The open-weight LLMs and our proposed model have shown very high accuracy for short input commands. In particular, the models were shown to reproduce the corresponding outputs on bash or the commercial tool Metasploit with very high precision. However, some models reproduce cases where some words used on the terminal were included. The proposed model was also able to implement the attacker's attack scenario correctly. The open-weight LLM models obtained similar but not identical responses based on the model's characteristics. Gemma3-4b produced an output that was similar to the correct answer; however, it omitted several sentences. In the case of gtp-oss:20b, it generated the correct answer independently. Qwen2-7b generated similar to the correct answer; some content is omitted. We compared whether each model produced proper output based on history. Notably, the proposed model generates accurate outputs by referencing existing input values, even if it slightly deviates from the historical reference range. Moreover, the proposed model produces answers that match the correct answer, assuming that the attacker utilizes and executes the Metasploit tool in the victim environment. In the case of GPT-OSS-20b, it generated similar content to the correct answer with hallucination, but in the case of Qwen2-7b, it constructed the correct answer in the same sentence.

5 Conclusion

This paper has proposed a MEAD framework that can generate output corresponding to an attacker's command in a Linux environment by utilizing the Mamba model. MEAD enables interaction in long-term sessions by contextualizing the attacker's sequential commands and environmental states, thereby generating output that maintains situational consistency, even without physically building a Linux system. Also,

MEAD addresses the memory bottleneck and delay issues associated with attention in Transformer-based approaches; it also offers an alternative to the learning instability and limited interaction range commonly found in RL techniques by providing a stable, data-driven generation method. Experiments have shown that MEAD displays high accuracy in terms of command-output consistency, and in particular, it confirms the possibility of continuous interaction by generating consistent output even under conditions where history length increases.

Although Mamba2 can offer additional efficiency gains, the current Mamba-based model already provides sufficient capability for state-consistent command–output generation in our setting [6]. We will incorporate Mamba2 to improve efficiency and training stability further.

6 Limitation

Experimental results have demonstrated that it is not possible to reproduce the long output flawlessly. In appendix, table 8 illustrates examples where the proposed framework failed. The proposed model tends to accept incorrect history as the history gets longer. We set the history length to 4, which is the length that the model can handle most effectively, but when the history length exceeded 5 or was below 3, it produced improper results. Additionally, it showed vulnerability to distributed attacks. If an attacker performs attacks sequentially and alternately based on two terminals, the proposed model fails to detect the changed episode and generates incorrect output. When the attacker interrupted the terminal on his own, the model did not accept the interrupt and output the command that was fully executed. Also, further verification is needed to assess responses to unusual environmental changes, such as elevated privilege, abnormal processes, and network fluctuations, which can occur in real attack scenarios. Future research will focus on developing a lightweight cyber deception environment that can be operated practically by expanding more diverse attack scenarios and resolving identification evasion issues.

Ethical Considerations

We took ethical issues very seriously when conducting research within the MEAD framework. This study inherently carries the potential to generate results that could lead to harmful or misused behavior. However, we have considered and responded in various ways to ensure that the research results are handled ethically and responsibly.

Stakeholder Considerations. MEAD is a deception framework that simulates attacker interactions; we acknowledge that MEAD can have different impacts on various stakeholders when extended outside of a research environment. From a security operator's perspective, inducing and observing attacker behavior can enhance detection and analysis capabilities. However, if the generated output is confused with actual system evidence, there is a risk that it will lead to incorrect incident analysis or response decisions. Conversely, from the service provider's perspective, interaction logs collected during the research process may contain operational or sensitive information, requiring data protection and access control. Additionally, from the viewpoint of external users or third parties, confusion can occur if a deceptive environment is mistaken for a legitimate service. Consequently, this study was conducted exclusively in an approved test environment, with the scope clearly defined and limited. To reduce these stakeholder risks, MEAD experiments and operations were conducted in a closed environment, with outputs and logs restricted to research and defense validation, and with a clear distinction made between artifacts and actual evidence when interpreting analysis results.

Responsible Misuse Concerns. Our research generally produces output corresponding to the attacker's command, but paradoxically, it has the problem that it can be shown as an operating service in certain environments. To address this issue, we restricted access during the implementation to only trusted researchers. Our MEAD is open source, but to prevent misuse, it does not include any functionality to directly induce or automate attack execution within the source code, and the output provided by the system is limited to observation, analysis, and defense verification. Additionally, we disclosed only the minimum configuration for reproduction, and settings with a high potential for misuse are excluded from the distribution or separated into a form requiring a separate approval process.

Human-Subject Research Potential Impact. Although MEAD itself is a software framework, it can have the character of human subjects research in that behavioral data from participants acting as attackers may be collected and analyzed in actual use. Particularly, command input patterns, session flow, and decision-making styles can be associated with the work characteristics and proficiency of an individual or organization, and logs can contain user identifiers, paths, host names, and internal resource information. Thus, this work collects data only under approved experimental conditions, and in environments where participants are present, the principle is to inform them in advance of the research purpose, scope of collection, retention period, and whether or not to anonymize the data. Additionally, the logs collected are limited to a minimum, identifiable information is masked or pseudonymized, and access is granted only to a limited number of researchers. When reporting results, we minimize the potential for infringement on the rights and privacy of participants and related entities by not including any details that could identify individuals or specific organizations.

Open Science

We are committed to the open science policy and have released our artifacts; the source code of MEAD framework and generated dataset sample at https://anonymous.4open.science/r/Who-Am-I-Command-To-D985. In this paper, we print the output corresponding to the attacker's command using the generated command-output dataset. However, from an ethical standpoint, there is a risk that the attackers' commands could be misused for malicious purposes; therefore, we release the full dataset under controlled access to address ethical concerns regarding potential misuse. Researchers can obtain access upon request for legitimate research purposes, subject to usage agreements that define the permitted scope.

References

[1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

[2] Shihab Ahmed, ABM Mohaimenur Rahman, Md Morshed Alam, and Md Sajidul Islam Sajid. Spade: Enhancing adaptive cyber deception strategies with generative ai and structured prompt engineering. In *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 01007–01013. IEEE, 2025.

[3] Robert A Bridges, Thomas R Mitchell, Mauricio Muñoz, and Ted Henriksson. Sok: Honeypots & llms, more than the sum of their parts? *arXiv preprint arXiv:2510.25939*, 2025.

[4] Barak Bringoltz, Elisha Halperin, Ran Feraru, Evgeny Blaichman, and Amit Berman. Clear: Command level annotated dataset for ransomware detection. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[5] Gideon Creech and Jiankun Hu. A semantic approach to host-based intrusion detection systems using contiguousand discontiguous system call patterns. *IEEE Transactions on Computers*, 63(4):807–819, 2013.

[6] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, pages 10041–10071. PMLR, 2024.

[7] Hien Do Hoang, Trong-Nghia To, Ngo Duc Hoang Son, Khoa Ngo-Khanh, Nguyen Tan Cam, and Van-Hau Pham. Hawkeyes: An intelligent honeypot allocation strategy for cyber deception using reinforcement learning. *Computer Networks*, page 111982, 2025.

[8] Javier Franco, Ahmet Aris, Berk Canberk, and A Selcuk Uluagac. A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems. *IEEE Communications Surveys & Tutorials*, 23(4):2351–2383, 2021.

[9] Tahani GAZDAR, Lara MURAD, Safana AL-JAHDALI, Fai ZWAWI, and Reman MANDILI. Tanglenet: An advanced cyber deception model based on reinforcement learning. *Romanian Journal of Information Technology and Automatic Control*, 35(1):23–36, 2025.

[10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First conference on language modeling*, 2024.

[11] Chongqi Guan, Guohong Cao, and Sencun Zhu. Honeyllm: Enabling shell honeypots with large language models. In *2024 IEEE Conference on Communications and Network Security (CNS)*, pages 1–9. IEEE, 2024.

[12] Martin Gubri, Dennis Ulmer, Hwaran Lee, Sangdoo Yun, and Seong Joon Oh. Trap: Targeted random adversarial prompt honeypot for black-box identification. In *ACL (Findings)*, 2024.

[13] Kevin Hobert, Charles Lim, and Eka Budiarto. Semantic similarities for honeypot collected linux shell commands. In *2023 11th International Conference on Information and Communication Technology (ICoICT)*, pages 172–177. IEEE, 2023.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

[15] Niclas Ilg, Paul Duplys, Dominik Sisejkovic, and Michael Menth. A survey of contemporary open-source honeypots, frameworks, and tools. *Journal of Network and Computer Applications*, 220:103737, 2023.

[16] Amir Javadpour, Forough Ja'fari, Tarik Taleb, Mohammad Shojafar, and Chafika Benzaïd. A comprehensive survey on cyber deception techniques to improve honeypot performance. *Computers & Security*, page 103792, 2024.

[17] Mateusz Kazimierczak, Nuzaira Habib, Jonathan H Chan, and Thanyathorn Thanapattheerakul. Impact of ai on the cyber kill chain: A systematic review. *Heliyon*, 10(24), 2024.

[18] Alexander Kedrowitsch, Danfeng Yao, Gang Wang, and Kirk Cameron. A first look: Using linux containers for deceptive honeypots. In *Proceedings of the 2017 Workshop on Automated Decision Making for Active Cyber Defense*, pages 15–22, 2017.

[19] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *EMNLP 2018*, page 66, 2018.

[20] Changsong Li, Ning Zhao, and Hao Wu. Multiple deception resources deployment strategy based on reinforcement learning for network threat mitigation. *Scientific Reports*, 15(1):16830, 2025.

[21] Pedro Beltrán López, Manuel Gil Pérez, and Pantaleone Nespoli. Cyber deception: State of the art, trends and open challenges. *arXiv preprint arXiv:2409.07194*, 2024.

[22] Zhuo Lu, Cliff Wang, and Shangqing Zhao. Cyber deception for computer and network security: Survey and challenges. *arXiv preprint arXiv:2007.14497*, 2020.

[23] Stephanie Milani, Weiran Shen, Kevin S Chan, Sridhar Venkatesan, Nandi O Leslie, Charles Kamhoua, and Fei Fang. Harnessing the power of deception in attack graph-based security games. In *International conference on decision and game theory for security*, pages 147–167. Springer, 2020.

[24] Iyatiti Mokube and Michele Adams. Honeypots: concepts, approaches, and challenges. In *Proceedings of the 45th annual ACM Southeast Conference*, pages 321–326, 2007.

[25] Paolo Notaro, Soroush Haeri, Jorge Cardoso, and Michael Gerndt. Command-line risk classification using transformer-based neural architectures. *arXiv preprint arXiv:2412.01655*, 2024.

[26] Hakan T Otal and M Abdullah Canbaz. Llm honeypot: Leveraging large language models as advanced interactive honeypot systems. In *2024 IEEE Conference on Communications and Network Security (CNS)*, pages 1–6. IEEE, 2024.

[27] Jeffrey Pawlick, Edward Colbert, and Quanyan Zhu. Modeling and analysis of leaky deception using signaling games with evidence. *IEEE Transactions on Information Forensics and Security*, 14(7):1871–1886, 2018.

[28] Niels Provos et al. A virtual honeypot framework. In *USENIX Security Symposium*, volume 173, pages 1–14, 2004.

[29] Jarrod Ragsdale and Rajendra V Boppana. On designing low-risk honeypots using generative pre-trained transformer models with curated inputs. *IEEE Access*, 11:117528–117545, 2023.

[30] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 1715–1725, 2016.

[31] Muris Sladić, Veronica Valeros, Carlos Catania, and Sebastian Garcia. Llm in the shell: Generative honeypots. In *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 430–435. IEEE, 2024.

[32] Muris Sladić, Veronica Valeros, Carlos Catania, and Sebastian Garcia. Vellmes: A high-interaction ai-based deception framework. In *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 671–679. IEEE, 2025.

[33] Lance Spitzner. *Honeypots: tracking hackers*. Addison-Wesley Longman Publishing Co., Inc., 2002.

[34] Shreyas Srinivasa, Jens Myrup Pedersen, and Emmanouil Vasilomanolakis. Gotta catch'em all: a multi-stage framework for honeypot fingerprinting. *Digital Threats: Research and Practice*, 4(3):1–28, 2023.

[35] Zarrin Tasnim Sworna, Zahra Mousavi, and Muhammad Ali Babar. Nlp methods in host-based intrusion detection systems: A systematic review and future directions. *Journal of Network and Computer Applications*, 220:103761, 2023.

[36] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[37] Qwen Team et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2(3), 2024.

[38] Christopher Theisen, Nuthan Munaiah, Mahran Al-Zyoud, Jeffrey C Carver, Andrew Meneely, and Laurie Williams. Attack surface definitions: A systematic literature review. *Information and Software Technology*, 104:94–103, 2018.

[39] Alexander Vetterl. Honeypots in the age of universal attacks and the internet of things. Technical report, University of Cambridge, Computer Laboratory, 2020.

[40] Alexander Vetterl and Richard Clayton. Bitter harvest: Systematically fingerprinting low-and medium-interaction honeypots at internet scale. In *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[41] Aaron R Voelker and Chris Eliasmith. Improving spiking dynamical networks: Accurate delays, higher-order synapses, and time cells. *Neural computation*, 30(3):569–609, 2018.

[42] Gérard Wagener, Radu State, Alexandre Dulaunoy, and Thomas Engel. Self adaptive high interaction honeypots driven by game theory. In *Symposium on Self-Stabilizing Systems*, pages 741–755. Springer, 2009.

[43] Cliff Wang and Zhuo Lu. Cyber deception: Overview and the road ahead. *IEEE Security & Privacy*, 16(2):80–85, 2018.

[44] Xiao Wang, Shiao Wang, Yuhe Ding, Yuehang Li, Wentao Wu, Yao Rong, Weizhe Kong, Ju Huang, Shihao Li, Haoxiang Yang, et al. State space model for new-generation network alternative to transformers: A survey. *arXiv preprint arXiv:2404.09516*, 2024.

[45] Ziyang Wang, Jianzhou You, Haining Wang, Tianwei Yuan, Shichao Lv, Yang Wang, and Limin Sun. Honeygpt: breaking the trilemma in terminal honeypots with large language model. *arXiv preprint arXiv:2406.01882*, 2024.

[46] Usman Wushishi, Altaf Hussain, Muhammad Imran Khalid, Nasir Hussain, Mona Jamjoom, and Zahid Ullah. D3o-iiot: deep reinforcement learning-driven dynamic deception orchestration for industrial iot security. *Scientific Reports*, 2025.

[47] Yueyue Xu, Yuewei Chen, Lin Wang, Zhaoyang Cheng, and Xiaoming Hu. Optimal honeypot ratio and convergent fictitious-play learning in signaling games for cps defense. *arXiv preprint arXiv:2507.11113*, 2025.

[48] Li Zhang and Vrizlynn LL Thing. Three decades of deception techniques in active cyber defense-retrospect and outlook. *Computers & Security*, 106:102288, 2021.

[49] Hengye Zhu, Mengxiang Liu, Binbin Chen, Xin Che, Peng Cheng, and Ruilong Deng. Honeyjudge: A plc honeypot identification framework based on device memory testing. *IEEE Transactions on Information Forensics and Security*, 19:6028–6043, 2024.

[50] Zhongyu Zhuang, Ziran Liang, Yanghui Rao, Haoran Xie, and Fu Lee Wang. Out-of-vocabulary word embedding learning based on reading comprehension mechanism. *Natural Language Processing Journal*, 5:100038, 2023.

[51] Valdemar Švábenský, Jan Vykopal, Pavel Seda, and Pavel Čeleda. Dataset of shell commands used by participants of hands-on cybersecurity training. *Data in Brief*, 38:107398, 2021.

A Experimental Metrics

EM measures whether the output predicted by the model is precisely the same as the correct sequence. EM is the most stringent metric, verifying that the model accurately reproduces the configuration between the predicted sequence and the terminal output. Equation 9 represents the formula for calculating the average EM based on the prediction $\hat{y}_i$ for sample i and the correct answer y_i. For each sample i, we define it as one if the prediction $\hat{y}_i$ and y_i are entirely identical, and zero otherwise. We calculated how much of the output matches the entire dataset based on the average value for all samples N. In the case of EM, it is challenging to represent partial quality with EM independently, as even slight variations, such as spaces and line breaks, can result in it being assessed as incorrect. Therefore, we interpret EM as a completely identical reproduction ratio, and analyzed together with other similarity-based indices.

$$EM_i = \begin{cases} 1 & if\ \hat{y}_i = y_i \\ 0 & otherwise \end{cases}$$
$$EM = \frac{1}{N}\sum_{i=1}^{N} EM_i \tag{9}$$

Char-F1 compares the sequence predicted by the model and the correct sequence character by character and calculates the F1-score for each character. Char-F1 effectively measures the extent of partial matches in terminal outputs that include various string patterns like special characters, paths, and hashes. If the predicted sequence and the correct sequence are respectively called $C(\hat{y})$ and $C(y)$, then character-level precision and recall can be calculated based on frequency. Also, Char-F1 can be derived based on character-level precision and recall.

Additionally, we utilized the ROUGE family of metrics to quantify the degree of n-gram overlap and structural similarity between the predicted output and the correct sequence. ROUGE is an index used to evaluate the performance of natural language generation models, including automatic text summarization and machine translation, within natural language processing. We utilized the F1-score from ROUGE-1, ROUGE-2, and ROUGE-L. ROUGE-1 and ROUGE-2 measure precision, recall, and F1-score by evaluating n-gram overlaps from token sequences. When the n-gram multisets for the predicted and correct values are $G_n(\hat{Y}), G_n(Y)$, respectively, the frequency values of n-gram g can be defined as $count_{\hat{Y}}(g), count_Y(g)$, respectively. Then, Overlap can be expressed as equation 10.

	Attacker	Victim 1	Victim 2	Victim 3	Victim 4
OS	Kali Linux 2025.1	Ubuntu 20.04	Ubuntu 22.04	Ubuntu 24.04.3	Debian
Linux Kernel	-	5.15.0-139-generic	6.8.0-90-generic	6.14.0-36-generic	6.1.0-34-amd64
Web Server	-	Apache 2.4.41	Apache 2.4.52	Apache 2.4.58	2.4.62
SSH	OpenSSH_9.9p2	OpenSSH_8.2p1	OpenSSH_8.9p1	OpenSSH_9.6p1	OpenSSH_9.2p1
FTP	-	vsftpd 3.0.5	vsftpd 3.0.5	vsftpd 3.0.5	vsftpd 3.0.3
Database	-	MySQL 8.0.41	MySQL 8.0.41	MySQL 8.0.44	MySQL 8.4.5
SNMP	-	NET-SNMP 5.8	NET-SNMP 5.9.1	NET-SNMP 5.9.4.pre2	NET-SNMP 5.9.3
Telnet	-	0.17-41.2build1	0.17-44build1	0.17+2.5-3ubuntu4	0.17+2.4-2+deb12u1
Metasploit	6.4.50-dev	-	-	-	-

Table 4: VM configurations

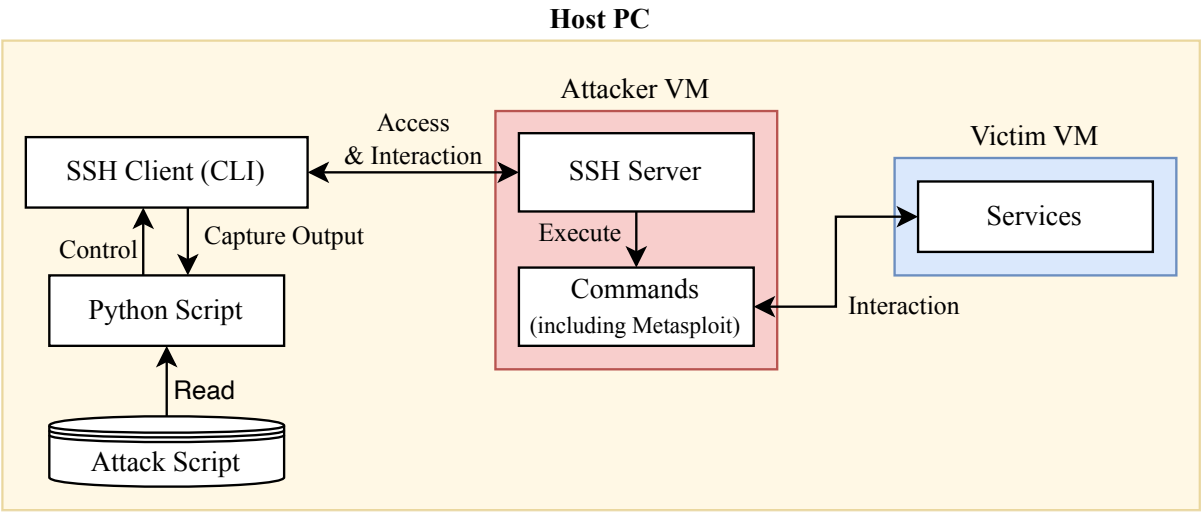


Figure 6: The structure of the cyber attack emulation environment. The host machine where the emulation operates consists of an attacker VM, a victim VM, and an attack script. The host machine controls an attacker VM via SSH to execute scripted attacks against a victim VM and collects command outputs.

$$overlap_n(\hat{Y}, Y) = \sum_g \min(count_{\hat{Y}}(g),\ count_Y(g)) \tag{10}$$

Following the equation 11, precision P_n and recall R_n can be derived based on overlap, and the F1-score can be calculated based on precision and recall.

$$\begin{aligned} P_n &= \frac{\text{overlap}_n(\hat{Y}, Y)}{\sum_{g \in G_n(\hat{Y})} \text{count}_{\hat{Y}}(g)} \\ R_n &= \frac{\text{overlap}_n(\hat{Y}, Y)}{\sum_{g \in G_n(Y)} \text{count}_Y(g)} \\ \text{ROUGE-N}_{F1} &= \frac{2P_nR_n}{P_n + R_n} \end{aligned} \tag{11}$$

ROUGE-L allows for numerical assessment of sentence structural similarity based on the length of the Longest Common Subsequence (LCS) between the predicted and correct values. As shown in Equation 12, let the length of the LCS between the prediction and the reference be denoted by $LCS(\hat{Y}, Y)$. Based on this quantity, we compute precision and recall, and then obtain the ROUGE-L F1-score.

$$\begin{aligned} P_L &= \frac{\text{LCS}(\hat{Y}, Y)}{|\hat{Y}|} \\ R_L &= \frac{\text{LCS}(\hat{Y}, Y)}{|Y|} \\ \text{ROUGE-L}_{F1} &= \frac{2P_LR_L}{P_L + R_L} \end{aligned} \tag{12}$$

B Linux Command-Output Dataset

To train the proposed framework, we constructed a dataset comprising attack command-output sequences generated by

Algorithm 1 Pseudo codes for constructing the Linux command dataset

```
Host Machine H, Attacker VM A, Python Script P
for Victim VM V in the listed victim machines do
    for Attack Script AS in the collected attack scripts do
        H initializes A and V
        H runs A
        H runs V
        P creates SSH connection from H to A
        for Attack Command AC in AS do
            P executes AC on A via SSH
            P captures output from A via SSH
            P stores the output with environment states on H
        end for
        P terminates the SSH connection
        H shuts down A and V
    end for
end for
```

an attacker in a Kali Linux environment.

The attack command-output sequences were obtained by emulating the attacker's activity across two Virtual Machines (VMs): the attacker and one of four distinct victim VMs, rotated. Table 4 shows the configurations for the attacker and the four victim VMs, including their OSs and installed services.

The attacker's attack scripts were derived from diverse research sources, including Valdemar et al. [51], which contain shell commands from cybersecurity training courses. We used a Python script that emulates an attacker's activity, executes attack shell command sequences, and captures the output from each shell command.

Figure 6 illustrates the structure of the emulation environment. The attacker and victim VMs are hosted on the host machine. The host machine connects to the attacker VM via an SSH Client (CLI) controlled by a Python script. The Python script reads the attack scripts and executes attack sequences on the attacker VM, targeting the selected victim VM. The Python script captures and records the output generated by each individual command in an attack script. Algorithm 1 outlines the procedure for dataset construction described above, in a sequential manner.

Using the emulation environment, we collected 21,108 command-output pairs for each of the four victim VMs across 267 attack sequences. In total, we constructed a dataset comprising 84,432 command-output pairs. Table 5 shows the breakdown of command counts between the two categories.

command category	count	description
bash command	64,332	Common Linux commands
attack tool command	20,100	Commands from Metasploit

Table 5: Number of command-output pairs in the dataset.

C Large Language Models in Experiment

Gemma3-4b. The Gemma3-4b model is an open-weight, decoder-only Transformer-based language model from the Gemma 3 family, comprising approximately 4 billion parameters, and is designed with an emphasis on efficiency and scalability.

GPT-OSS-20b. The Generative Pre-trained Transformer Open Source Software (GPT-OSS) model is an open-weight Transformer-based language model developed by OpenAI under the Apache 2.0 license. The GPT-OSS-20b model consists of 24 layers and has approximately 21B parameters. Furthermore, the GPT-OSS 20b model uses a Mixture-of-Experts architecture to improve computational efficiency and reduce memory requirements.

Qwen2-7b. Qwen2-7b model is an open-weight Transformer-based language model released by the Alibaba. The Qwen2-7b model achieves strong performance across a broad set of benchmarks, including language understanding and generation, multilingual tasks, coding, mathematics, and reasoning.

D Terminologies

Interaction. Interaction is a closed-loop exchange where an attacker issues a shell command, and MEAD returns a terminal output in real time. The interaction is maintained across a session by reflecting command history and the current context so that the deception remains consistent.

Kali Linux. Kali Linux is a Debian-based Linux distribution designed for penetration testing and security assessment, providing a large collection of preinstalled offensive security tools.

Model	Metrics	History length									
		1	2	3	**4**	5	6	7	8	9	10
Gemma3-4b	Exact Match	0.513	0.503	0.503	**0.503**	0.508	0.503	0.508	0.508	0.513	0.508
	Char-F1	0.745	0.738	0.740	**0.737**	0.737	0.736	0.743	0.740	0.747	0.742
	ROUGE-1	0.665	0.656	0.657	**0.656**	0.659	0.655	0.659	0.660	0.667	0.661
	ROUGE-2	0.643	0.628	0.634	**0.628**	0.632	0.627	0.631	0.638	0.638	0.633
	ROUGE-L	0.663	0.654	0.655	**0.654**	0.657	0.653	0.658	0.658	0.665	0.659
	Avg. time (Sec)	3.293	3.271	3.262	**3.268**	3.275	3.265	3.273	3.295	3.269	3.278
GPT-OSS-20b	Exact Match	0.450	0.450	0.461	**0.456**	0.456	0.445	0.445	0.456	0.450	0.456
	Char-F1	0.539	0.539	0.538	**0.537**	0.531	0.552	0.534	0.531	0.526	0.554
	ROUGE-1	0.518	0.515	0.524	**0.517**	0.516	0.529	0.514	0.513	0.508	0.536
	ROUGE-2	0.560	0.566	0.570	**0.551**	0.558	0.571	0.551	0.551	0.557	0.581
	ROUGE-L	0.518	0.515	0.524	**0.517**	0.516	0.529	0.514	0.513	0.508	0.536
	Avg. time (Sec)	19.872	18.897	20.198	**18.980**	21.510	20.498	18.946	19.280	20.507	20.841
Qwen2-7b	Exact Match	0.335	0.319	0.319	**0.319**	0.330	0.340	0.314	0.314	0.319	0.319
	Char-F1	0.691	0.680	0.686	**0.686**	0.694	0.682	0.683	0.686	0.676	0.679
	ROUGE-1	0.586	0.580	0.587	**0.584**	0.590	0.583	0.580	0.581	0.580	0.578
	ROUGE-2	0.471	0.457	0.462	**0.459**	0.484	0.480	0.450	0.454	0.464	0.453
	ROUGE-L	0.584	0.578	0.585	**0.582**	0.589	0.582	0.579	0.579	0.578	0.576
	Avg. time (Sec)	3.325	3.316	3.338	**3.307**	3.310	3.297	3.322	3.322	3.326	3.331
MEAD framework	Exact Match	0.852	0.908	0.934	**0.939**	0.924	0.903	0.847	0.816	0.587	0.250
	Char-F1	0.939	0.953	0.954	**0.954**	0.948	0.943	0.914	0.889	0.686	0.362
	ROUGE-1	0.920	0.940	0.940	**0.941**	0.931	0.924	0.876	0.839	0.611	0.268
	ROUGE-2	0.921	0.954	0.943	**0.939**	0.939	0.926	0.875	0.832	0.603	0.261
	ROUGE-L	0.918	0.937	0.940	**0.940**	0.930	0.923	0.875	0.838	0.610	0.266
	Avg. time (Sec)	1.044	1.144	1.213	**1.190**	1.330	1.335	1.407	1.476	1.537	1.696

Table 6: Impact of history length on inference performance.

Linear Time-Invariant systems. LTI generates an output signal from any input signal, adhering to the principles of linearity and time invariance.

Longest Common Subsequence. LCS is the longest sequence of tokens that appears in the same relative order in two strings, without requiring contiguity.

Metasploit. Metasploit is a widely used penetration testing framework that provides modular exploits, payloads, and auxiliary tools for validating and executing attack scenarios.

Selective fine-tuning. Selective fine-tuning is a parameter-efficient adaptation strategy that updates only a targeted subset of model parameters. Selective fine-tuning reduces training cost and overfitting risk while preserving the pretrained model's general capabilities, enabling domain adaptation with limited compute and data.

Tokenization. Tokenization is the process of converting raw text into discrete tokens that a language model can process as integer IDs. The selection of the tokenizer and special tokens influences the representation of commands, paths, and outputs, which directly affects generation fidelity and evaluation consistency.

Transformer. Transformer is a neural sequence model built around self-attention, enabling strong contextual modeling by letting each token attend to other tokens in the sequence.

Virtual Machine. A VM is a software-based emulation of a computer that runs an OS and applications in an isolated environment on shared physical hardware.

Attacker's command	Model	Inference output	Goal output
ssh alice@192.168.56.105	MEAD	alice@ubuntu	alice@ubuntu
	Gemma3-4b	alice@ubuntu	
	GPT-OSS-20b	alice@ubuntu	
	Qwen2-7b	alice@ubuntu$	
set RHOST 192.168.56.105	MEAD	RHOST => 192.168.56.105	RHOST => 192.168.56.105
	Gemma3-4b	RHOST => 192.168.56.105	
	GPT-OSS-20b	RHOST => 192.168.56.105	
	Qwen2-7b	RHOST => 192.168.56.105	
nmap -sV 192.168.56.105	MEAD	Starting Nmap 7.95 (https://nmap.org) at 2025-12-10 19:05 KST setup_target: failed to determine route to 192.168.56.105 WARNING: No targets were specified, so 0 hosts scanned. Nmap done: 0 IP addresses (0 hosts up) scanned in 0.07 seconds	Starting Nmap 7.95 (https://nmap.org) at 2025-12-10 19:05 KST setup_target: failed to determine route to 192.168.56.105 WARNING: No targets were specified, so 0 hosts scanned. Nmap done: 0 IP addresses (0 hosts up) scanned in 0.07 seconds
	Gemma3-4b	Starting Nmap 7.95 (https://nmap.org) at 2025-12-10 19:05 KST setup_target: failed to determine route to 192.168.56.105	
	GPT-OSS-20b	Starting Nmap 7.95 (https://nmap.org) at 2025-12-10 19:05 KST Nmap scan report for 192.168.56.105 Host is up (0.000078s latency). Not shown: 997 closed tcp ports (reset) PORT STATE SERVICE VERSION 22/tcp open ssh OpenSSH 9.6p1 Ubuntu 3ubuntu13.14	
	Qwen2-7b	Starting Nmap 7.95 (https://nmap.org) at 2025-12-10 19:05 KST	
history	MEAD	1 sudo su 2 ping 192.168.56.105 3 ssh alice@192.168.56.105 4 ssh alice@192.168.56.105 5 exit 6 nmap -sV 192.168.56.105	1 sudo su 2 ping 192.168.56.105 3 ssh alice@192.168.56.105 4 ssh alice@192.168.56.105 5 exit 6 nmap -sV 192.168.56.105
	Gemma3-4b	sudo su ping 192.168.56.105 ssh alice@192.168.56.105 exit nmap -sV 192.168.56.105	
	GPT-OSS-20b	1 sudo su 2 ssh alice@192.168.56.105 3 ssh alice@192.168.56.105 4 nmap -sV 192.168.56.105	
	Qwen2-7b	1 sudo su 2 ping 192.168.56.105 3 ssh alice@192.168.56.105 4 nmap -sV 192.168.56.105	
use auxiliary/scanner/ssh/libssh_auth_bypass => run	MEAD	[*] 192.168.56.105:22 - Attempting authentication bypass [*] Scanned 1 of 1 hosts (100% complete) [*] Auxiliary module execution completed	[*] 192.168.56.105:22 - Attempting authentication bypass [*] Scanned 1 of 1 hosts (100% complete) [*] Auxiliary module execution completed
	Gemma3-4b	[*] 192.168.56.105:22 - Attempting authentication bypass [*] Auxiliary module execution completed	
	GPT-OSS-20b	[*] 192.168.56.105:22 - Attempting authentication bypass [+] Host appears vulnerable to libssh auth bypass [*] Auxiliary module execution completed	
	Qwen2-7b	[*] 192.168.56.105:22 - Attempting authentication bypass [*] Scanned 1 host (100% complete) [*] Auxiliary module execution completed	

Table 7: Example of inference results on each model.

Attacker's command	Inference output	Goal output	Issue
nmap -sV -p 192.168.56.105 options	setup_target_cache: Warp up to 2 targets simultaneously (caching) use_root_path: Use the root path as the main path unmask-authentication-question: Unmask the authentication question unmask-password: Unmask the password field unmask-result: Unmask the result field usermod-S-dname: Change user name to dname usermod-L-group: Change user group to group name usermod-V-name: Change user name to realname usermod-f: Force the action even if the destination is not a valid place to move to usermod-i: Disable login shortcuts usermod-n: Never check against a secret list usermod-N: Never change the user's primary group usermod-o: Output information for logs usermod-s: Enable interactive features usermod-U: User name to use usermod-v: Verbose mode	OS DETECTION: -O: Enable OS detection –osscan-limit: Limit OS detection to promising targets –osscan-guess: Guess OS more aggressively TIMING AND PERFORMANCE: Options which take <time> are in seconds, or append 'ms' (milliseconds), 's' (seconds), 'm' (minutes), or 'h' (hours) to the value (e.g. 30m). probe round trip time. –min-rate < *number* >: Send packets no slower than <number> per second –max-rate < *number* >: Send packets no faster than <number> per second	Refer to incorrect history
ssh alice@192.168.56.105	alice@192.168.56.105's password:	Are you sure you want to continue connecting (yes/no/[fingerprint])?	Distributed attack
ssh bob@192.168.56.106	bob@ubuntu	alice@ubuntu	User's Interrupt

Table 8: Example of fail inference results on MEAD framework.